

Windows Fundamentals & Dual CLI Arsenal

Architecture Essentials, Kernel Rings, Filesystem Internals, SAM Database & Dual CMD/PowerShell Operations

Instructor: Yahya Zaghloul

Track: Systems & Endpoint Security

Wargame: UnderTheWire Century

// 1. Windows Architecture: Dual-Mode Operation (Ring 3 vs. Ring 0)

Windows isolates system stability and memory integrity via two hardware-enforced CPU privilege rings:

- **User Mode (Ring 3):** Where user apps run (Chrome, Word, cmd.exe, powershell.exe). Each process operates in an isolated virtual address space. If an app crashes, the kernel and other apps stay completely safe.
- **Kernel Mode (Ring 0):** The core OS engine (ntoskrnl.exe), HAL (hal.dll), and device drivers (.sys). Unrestricted hardware and RAM access. A single unhandled exception or crash triggers a Blue Screen of Death (BSOD)!
- **The Syscall Bridge:** Apps cannot access hardware directly: User App -> kernel32.dll -> ntdll.dll (SYSCALL CPU instruction) -> ntoskrnl.exe.

SECURITY TAKEAWAY: RING 0 GOD-MODE & BYOVD

Attackers target Ring 0 to blind EDR agents. The Bring Your Own Vulnerable Driver (BYOVD) technique exploits an old, legitimately signed driver with known flaws to execute rootkit code inside Ring 0 without triggering Driver Signature Enforcement.

// 2. The Windows Filesystem (C:\) & The 64-Bit Paradox

Windows organizes storage into drive letters (C:\, D:\) formatted with NTFS (New Technology File System):

- **C:\Windows\System32:** The 64-bit Core! Contains 64-bit OS tools (cmd.exe, powershell.exe, lsass.exe, svchost.exe).
- **C:\Windows\SysWOW64:** The 32-bit Legacy Layer! Stands for 'Windows 32-bit On Windows 64-bit' for backward compatibility.
- **C:\Program Files:** 64-bit installed software (protected by strict ACLs; standard users cannot write here without UAC).
- **C:\Program Files (x86):** Legacy 32-bit installed software running under the WoW64 emulation subsystem.
- **C:\Users\<Username>:** User profile space containing Desktop, Documents, Downloads, and AppData.

// 3. Threat Landscape: Where Malware Hides in the Filesystem

Because attackers cannot write to Program Files or System32 without UAC prompts, 90%+ of malware targets:

- **AppData\Local\Temp (%TEMP%):** The #1 Dropper Haven! World-writable by standard users AND allows binary execution by default. Initial phishing attachments drop and unpack payloads here.
- **AppData\Roaming (%APPDATA%):** Follows Active Directory domain roaming profiles. Stores browser cookies, saved passwords, Telegram tokens.
- **C:\Users\Public (%PUBLIC%):** Writable by EVERY user on the machine. Classic staging ground for cross-user privilege escalation.
- **Alternate Data Streams (ADS):** NTFS feature: type evil.exe > notes.txt:malware.exe (Audit with dir /r or Get-Item -Stream *).
- **Anti-Forensics (Timestomping):** Attackers forge CreationTime to blend with system files: (Get-Item file.exe).CreationTime.

// 4. SAM Architecture & Windows Authentication Mechanics

The Security Account Manager (SAM) is Windows' local security database located at C:\Windows\System32\config\SAM:

- **The Kernel Lock:** When Windows boots, the kernel locks the SAM exclusively under SYSTEM. Even local Administrator receives Access Denied while Windows is running!
- **NTLM Password Hashes:** Windows NEVER stores plaintext passwords in the SAM! It stores one-way cryptographic NTLM hashes (MD4-based) encrypted with the SYSTEM registry key.
- **The Authentication Flow:** winlogon.exe captures credentials -> lsass.exe queries SAM database -> validates NTLM hash -> mints Access Token.
- **Forensics & Attack Vectors:** Attackers dump SAM via Volume Shadow Copies (vssadmin create shadow /for=C:) or extract credentials directly from LSASS memory using Mimikatz (requires SeDebugPrivilege).

// 5. Security Identifiers (SIDs) & Access Token Anatomy

Windows evaluates immutable digital SIDs (S-1-5-21-...-RID), never human username strings like 'Alice':

- **RID 500 (Built-in Administrator):** Absolute god-mode on local system. Even if renamed to 'Guest', its SID always ends in 500!
- **RID 501 (Built-in Guest):** Default guest account for unauthenticated access. Disabled by default in modern Windows.
- **RID 1000+ (Custom Users):** Assigned sequentially to custom user accounts created after OS installation (Alice=1001, Bob=1002).
- **Access Token Recon:** whoami /user (view SID), whoami /priv (active token privileges), whoami /groups (group memberships).

// 6. CMD vs. PowerShell: Strings vs. Objects

THE BIG DIFFERENCE: PAPER RECEIPT VS. SMART TOY

CMD (1987) outputs dumb plain text lines (like a printed paper receipt). Slicing and parsing strings breaks easily. PowerShell (2006) outputs live .NET objects with clickable properties (like a smart toy with buttons: .Name, .Id, .Status)!

PowerShell Syntax: Standardized Verb-Noun Grammar

Commands follow an Action-Target rule: Get-Process (query), Stop-Service (action), New-Item (create), Get-Content (read).

Friendly Aliases (Your Existing Muscle Memory Works!)

Alias (What You Type)	Real PowerShell Cmdlet	Functionality
ls or dir	Get-ChildItem	Lists directory contents and files
cat or type	Get-Content	Reads and outputs file content
ps or tasklist	Get-Process	Lists all active running processes
clear or cls	Clear-Host	Clears the terminal screen buffer
cp or copy	Copy-Item	Copies files or directories
rm or del	Remove-Item	Deletes files or directories

PowerShell Command Structure & Parameters

// PowerShell Syntax Foundation

```
# Cmdlet syntax: Verb-Noun -Parameter Value
Get-ChildItem -Path C:\Users -Force -Recurse
# Help system: Get-Help <Cmdlet> -Examples
Get-Help Get-Process -Examples
```

// 7. Dual Command Arsenal: CMD vs. PowerShell Side-by-Side

In penetration testing, you often land on a basic CMD shell (web shells, exploit payloads) before escalating to PowerShell:

Reconnaissance, Navigation & System Info

Action	Command Prompt (CMD)	PowerShell (Modern Cmdlet)
Print Current Directory	cd (no args)	Get-Location (or: pwd, gl)
Change Directory	cd C:\Users & cd /d D:	Set-Location C:\Users (or: cd)
List Files (Inc. Hidden)	dir /a	Get-ChildItem -Force (or: ls, dir)
User & Hostname	whoami & hostname	whoami & \$env:COMPUTERNAME
Environment Variables	set	Get-ChildItem env:
IP & Network Config	ipconfig /all	Get-NetIPAddress (or: ipconfig)

File & Folder Operations

Action	Command Prompt (CMD)	PowerShell (Modern Cmdlet)
Create Directory / File	mkdir Lab & echo X > f.txt	New-Item -ItemType Directory Lab ; New-Item f.txt -Value X
Copy / Move File	copy a.txt b.txt & move a.txt C:\	Copy-Item a.txt b.txt ; Move-Item a.txt C:\Temp\
Delete File / Tree	del f.txt & rmdir /s /q Lab	Remove-Item f.txt ; Remove-Item Lab -Recurse -Force
Read File / Grep Search	type f.txt & findstr /i "pwd" *	Get-Content f.txt ; Select-String -Pattern "pwd" *

Core Windows Engines: Understanding Services & The Registry

- Windows Services:** Headless background daemons managed by SCM (services.exe). Boot before login under NT AUTHORITY\SYSTEM (god mode). Key targets for privilege escalation & persistence.
- The Windows Registry:** Centralized hierarchical database (Hives -> Keys -> Values) replacing Linux /etc. HKLM = Machine-wide (admin write-only). HKCU = Current User (user writable; Run keys persist malware!).

User, Privilege, Service & Registry Auditing

Task	Command Prompt (CMD)	PowerShell (Modern Cmdlet)
List Local Users & Groups	net user & net localgroup	Get-LocalUser ; Get-LocalGroup
Administrators Membership	net localgroup Administrators	Get-LocalGroupMember -Group "Administrators"
Active Token Privileges	whoami /priv	whoami /priv (Look for SeImpersonatePrivilege!)
Active Processes / Kill	tasklist & taskkill /PID 123 /F	Get-Process ; Stop-Process -Id 123 -Force
Services / Registry Run	sc query & reg query ...\Run	Get-Service ? Status -eq 'Running' ; Get-ItemProperty ...\Run

The Object Pipeline: Conveyor Belt Manipulation

```
// PowerShell Pipeline Power
# Filter with Where-Object (alias: ?): Get-Service | Where-Object Status -eq 'Running'
# Filter memory hogs: Get-Process | Where-Object { $_.WS -gt 100MB }
# Sort & Select top 3: Get-Process | Sort-Object -Descending WS | Select-Object -First 3 Name, Id, WS
# Count items / lines: (Get-ChildItem Desktop).Count ; Get-Content auth.log | Measure-Object -Line
```

// 8. Live Game: Team CMD vs. Team PowerShell (5 Fundamental Differences)

Comparison Area	Team CMD (Legacy 1987)	Team PowerShell (Modern .NET)
1. Data Pipeline	Raw Text Strings (dumb receipt; text parsing breaks)	Live .NET Objects (smart objects: .Name, .Id, .Status)
2. Command Grammar	Inconsistent DOS switches (dir /b, findstr /i, tasklist /svc)	Standardized Verb-Noun formula (Get-Process, New-Item)
3. Engine Power	Simple external binary launcher (cmd.exe)	Full .NET Runtime, Win32 API access & virtual PSDrives
4. Error Handling	Primitive %ERRORLEVEL% numeric exit codes	Structured OOP exceptions (\$Error array, try/catch blocks)
5. Security Guardrails	Zero guardrails (runs any .bat file without checks)	Execution Policies, Script Block Logging (4104), AMSI

// 9. Practical Assignment: UnderTheWire Century (01-10)

UnderTheWire is a progressive cybersecurity wargame played directly over SSH. The password for each level is the answer to the previous level's challenge!

```
// UnderTheWire SSH Connection
# Connect via SSH from your terminal (PowerShell, CMD, or Linux):
ssh century1@century.underthewire.org -p 22
# Default password for Level 1: century1
```

Wargame Methodology & Rules of Engagement

- **Chained Authentication:** Progressive security challenge: the objective in Level N reveals the password string to unlock Level N+1 over SSH.
- **Authentic Muscle Memory:** No public walkthroughs or cheat sheets. Use native PowerShell cmdlets and pipeline filters learned today.
- **Documentation Primitives:** When stuck on syntax, use `Get-Help <Cmdlet> -Examples` or tab-completion to inspect cmdlet arguments.

Student Century 01-10 Challenge Tracker (Submit Your Progress)

Target Level	Challenge Objective	Your Discovered Flag / Cmdlet	Done?
Century 1 -> 2	Locate briefing file on Desktop & extract password	[Student to solve & enter flag]	[]
Century 2 -> 3	Determine file quantity in target folder	[Student to solve & enter flag]	[]
Century 3 -> 4	Discover & inspect hidden directory assets	[Student to solve & enter flag]	[]
Century 4 -> 5	Pattern match text in files (hunt credentials)	[Student to solve & enter flag]	[]
Century 5 -> 6	Inspect OS system environment variables	[Student to solve & enter flag]	[]
Century 6 -> 10	Sort objects, count lines & pipeline filtering	[Student to solve & enter flag]	[]

// 10. Local Machine 4-Point System Audit Checklist

Run this rapid 4-point security posture audit on your local Windows system to evaluate privilege boundaries:

```
// Local System Security Checklist
# 1. Identity & Privileges (Check your SID & Look for SeImpersonatePrivilege):
whoami /user ; whoami /priv
# 2. Local Administrators (Who holds god-mode on this machine?):
Get-LocalGroupMember -Group "Administrators"
# 3. Active Background Services (Check what is running as SYSTEM):
Get-Service | Where-Object Status -eq 'Running'
# 4. Startup Persistence (Check what boots automatically with Windows):
Get-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run'
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run'
```